

CRAWL DATA

1. CÀI ĐẶT MONGODB

MongoDB còn được gọi là Mongo, là một cơ sở dữ liệu tài liệu mã nguồn mở được sử dụng rộng rãi. MongoDB là cơ sở dữ liệu NoSQL. MongoDB không giống như các cơ sở dữ liệu quan hệ dựa trên SQL truyền thống như MySQL, Oracle,... MongoDB lưu trữ dữ liệu ở định dạng giống JSON (BSON) theo mặc định (BSON là Binary JSON). Bằng cách sử dụng BSON thay vì JSON thuần túy, kích thước của tài liệu có thể được giảm đáng kể. Bởi vì chúng lưu trữ dữ liệu trong các cặp khóa-giá trị, các DB này rất hiệu quả cho các mục đích đọc hoặc truy vấn.

Cập nhật hệ thống

Để đảm bảo các thư viện cần thiết của MongoDB đều được cập nhật

```
$ sudo apt update -y
```

```
$ sudo apt dist-upgrade -y
```

Cài đặt các gói thư viện bắt buộc

```
$ sudo apt install gnupg
```

Cài đặt thư viện libssl1

```
$ wget http://archive.ubuntu.com/ubuntu/pool/main/o/openssl/libssl1.1_1.1.0g-2ubuntu4_amd64.deb
```

```
$ sudo dpkg -i libssl1.1_1.1.0g-2ubuntu4_amd64.deb
```

Cài đặt MongoDB

Vào <https://www.mongodb.com/docs/manual/tutorial/install-mongodb-on-ubuntu/> tìm hướng dẫn cài đặt *Install MongoDB Community Edition* và làm theo các bước.

Khởi động Mongo Service

```
$ sudo systemctl start mongod.service
```

Khởi động lại dịch vụ

```
$ sudo systemctl restart mongod
```

Tự khởi động cùng hệ thống

```
$ sudo systemctl enable mongod
```

Tắt tự khởi động cùng hệ thống

```
$ sudo systemctl disable mongod
```

Tắt dịch vụ

```
$ sudo systemctl stop mongod
```

Xem trạng thái dịch vụ

\$ sudo systemctl status mongod

```
tqkhai@khai-master:~$ sudo systemctl status mongod
● mongod.service - MongoDB Database Server
   Loaded: loaded (/lib/systemd/system/mongod.service; disabled; vendor prese
   Active: active (running) since Sun 2025-03-02 09:33:21 UTC; 1min 12s ago
     Docs: https://docs.mongodb.org/manual
    Main PID: 2946 (mongod)
      Memory: 88.1M
         CPU: 1.124s
    CGroup: /system.slice/mongod.service
            └─2946 /usr/bin/mongod --config /etc/mongod.conf

Mar 02 09:33:21 khai-master systemd[1]: Started MongoDB Database Server.
Mar 02 09:33:21 khai-master mongod[2946]: {"t":{"$date":"2025-03-02T09:33:21.91
lines 1-12/12 (END)
```

Sử dụng Mongo

\$ mongosh

Thoát khỏi Mongo

\$ exit

2. CRAWL DATA (sử dụng account thường, không phải hadoop)

Cài đặt lệnh pip

\$ sudo apt-get update

\$ sudo apt install python3-pip

Cập nhật phiên bản pip mới nhất

\$ sudo pip install --upgrade pip

Cài đặt thư viện Scrappy

\$ sudo -H pip install scrapy

Tạo project Scrappy bằng lệnh

\$ scrapy startproject name_project

Trong đó name_project là tên project muốn khởi tạo

\$ scrapy startproject demo_crawl

```
tqkhai@khai-master:~$ scrapy startproject demo_crawl
New Scrapy project 'demo_crawl', using template directory '/home/tqkhai/.local/lib/python3.10/site-packages/scrapy/templates/project', created in:
/home/tqkhai/demo_crawl

You can start your first spider with:
cd demo_crawl
scrapy genspider example example.com
tqkhai@khai-master:~$
```

Cấu trúc thư mục project

demo_crawl/

scrapy.cfg	# deploy configuration file
demo_scrapy/	# project's Python module, you'll import your code from here
__init__.py	
items.py	# project items definition file
middlewares.py	# project middlewares file
pipelines.py	# project pipelines file
settings.py	# project settings file
spiders/	# a directory where you'll later put your spiders
__init__.py	

Tạo một spider bằng lệnh trong thư mục project/spiders

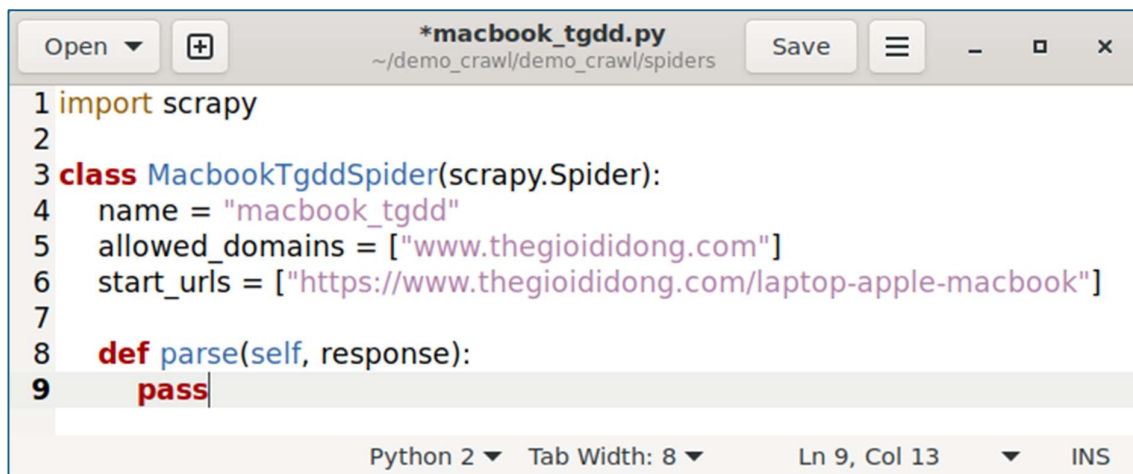
Spider dùng để tiến hành crawl dữ liệu từ website

```
$ scrapy genspider name_spider 'example.com'
```

Trong đó name_spider là tên của spider muốn tạo và “example.com” là tên miền của website muốn thu thập dữ liệu.

```
$ scrapy genspider macbook_tgdd www.thegioidadong.com/laptop-apple-macbook
```

Sau câu lệnh trên, một spider đã được tạo MacbookTgddSpider trong tập tin macbook_tgdd.py, sau đó di chuyển tập tin này vào thư mục spiders của project.



```
1 import scrapy
2
3 class MacbookTgddSpider(scrapy.Spider):
4     name = "macbook_tgdd"
5     allowed_domains = ["www.thegioidadong.com"]
6     start_urls = ["https://www.thegioidadong.com/laptop-apple-macbook"]
7
8     def parse(self, response):
9         pass
```

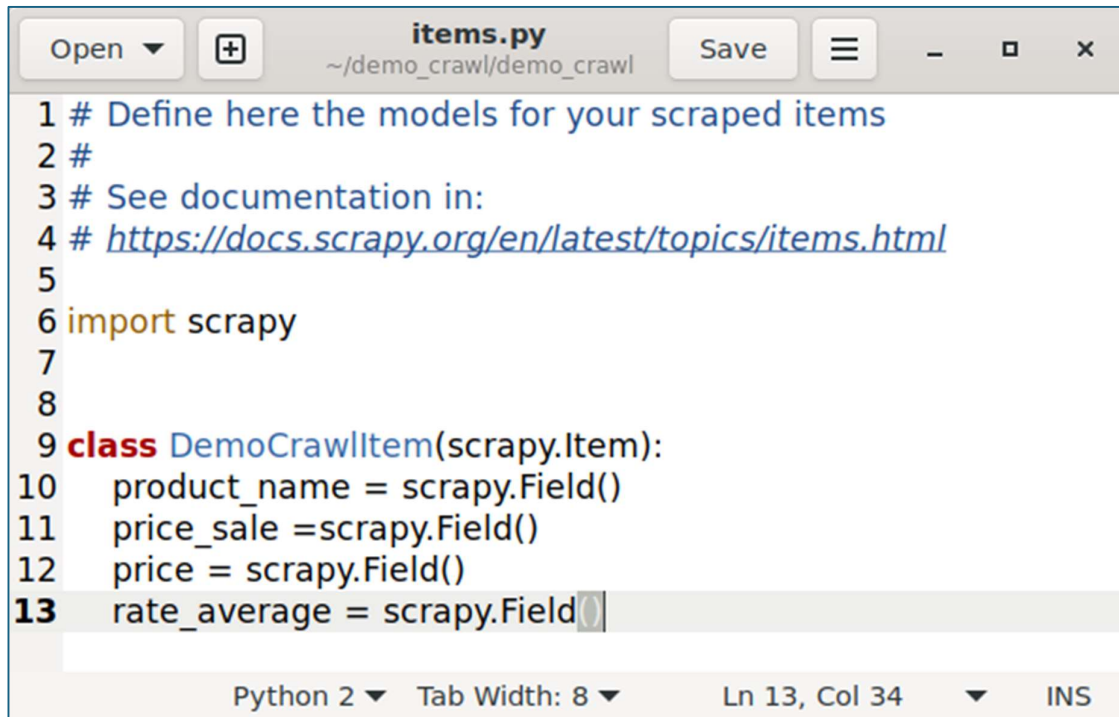
Trong tập tin trên:

- Con spider này có tên là macbook_tgdd sẽ được sử dụng để chạy lệnh lúc crawl.
- start_urls: đây là địa chỉ bắt đầu cho spider, có thể là một list các url tương ứng với domain trong allowed_domains.

- parse(): Chính là function sẽ viết code để điều khiển spider đi crawl những data mà mình muốn lấy về từ url phía trên.

Lựa chọn dữ liệu muốn crawl

Cần xác định trước data mà chúng ta muốn là những gì. Ở đây cần thống kê thị trường Macbook được bán ở thegioididong nên sẽ crawl tên sản phẩm, giá gốc, giá sale và lượng vote trung bình của mỗi sản phẩm macbook. Sau khi đã lên ý tưởng những item muốn crawl thì việc tiếp theo là define những item đó trong file items.py như sau:



```
1 # Define here the models for your scraped items
2 #
3 # See documentation in:
4 # https://docs.scrapy.org/en/latest/topics/items.html
5
6 import scrapy
7
8
9 class DemoCrawlItem(scrapy.Item):
10     product_name = scrapy.Field()
11     price_sale = scrapy.Field()
12     price = scrapy.Field()
13     rate_average = scrapy.Field()
```

Để crawl được đúng item mình cần thì cần phải chọn được đúng tới đối tượng đó trên DOM. Có thể sử dụng CSS Selectors hoặc XPath đều được, tuy nhiên mỗi cái đều có ưu nhược điểm riêng nên cần lựa chọn để sử dụng cho phù hợp.

Dùng CSS Selectors để trỏ tới những item mà mình muốn lấy, bật inspect lên và copy selector của những item đó như sau:

Viết thêm vào spiders/macbook_tgdd.py

```
import scrapy
from demo_crawl.items import DemoCrawlItem

class MacbookTgddSpider(scrapy.Spider):
    name = 'macbook_tgdd'
    allowed_domains = ['www.thegioididong.com']
    start_urls = ['https://www.thegioididong.com/laptop-apple-macbook/']

    def parse(self, response):
        # Request tới từng sản phẩm có trong danh sách các Macbook dựa vào href
        for item_url in response.css("li.item > a ::attr(href)").extract():
            # Nếu có sản phẩm thì sẽ gọi tới function parse_macbook
            yield scrapy.Request(response.urljoin(item_url), callback=self.parse_macbook)

        # nếu có sản phẩm kế tiếp thì tiếp tục crawl
        next_page = response.css("li.next > a ::attr(href)").extract_first()
        if next_page:
            yield scrapy.Request(response.urljoin(next_page), callback=self.parse)

    def parse_macbook(self, response):
        item = DemoCrawlItem()
        item['product_name'] = response.css(
            'div.rowtop > h1 ::text').extract_first() # Tên macbook
        out_of_stock = response.css('span.productstatus ::text').extract_first() # Tình trạng còn hàng hay không
        if out_of_stock:
            item['price'] = response.css(
                'strong.pricesell ::text').extract_first()
        else:
            item['price'] = response.css(
                'aside.price_sale > div.area_price.notapply > strong ::text').extract_first()
        # Check nếu có giảm giá khi mua online hay không
        discount_online = response.css('div.box-online.notapply').extract_first()
        if discount_online:
            item['price_sale'] = response.css(
                'aside.price_sale > div.box-online.notapply > div > strong ::text').extract_first()
        else:
            item['price_sale'] = response.css(
                'span.hisprice ::text').extract_first()
        item['rate_average'] = response.css('div.toprt > div.crt > div::attr(data-gpa)').extract_first()
        yield item
```

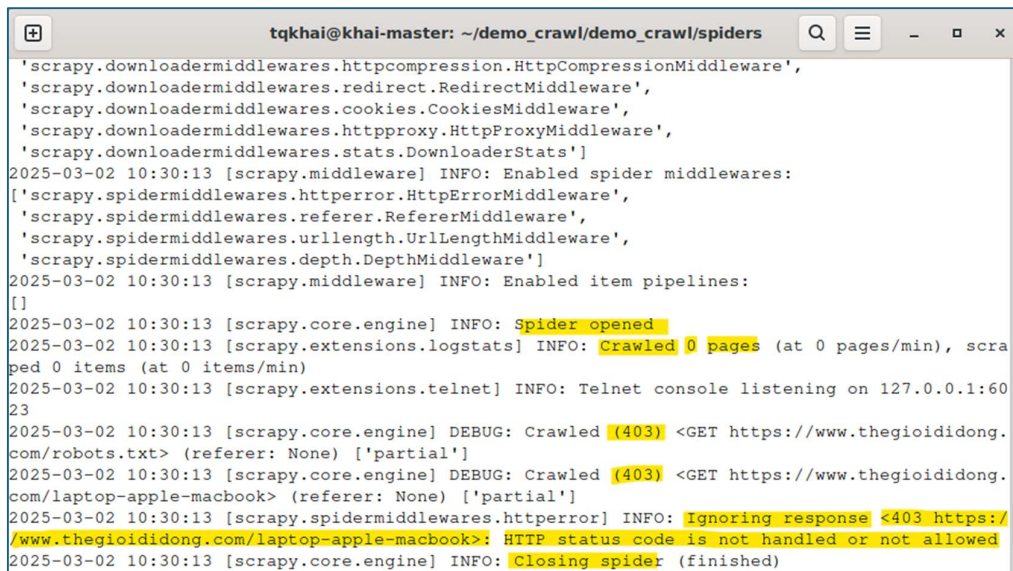
Hướng dẫn:

- **parse()**: Vì các dữ liệu của từng sản phẩm ở bên ngoài danh sách Macbook có thể chưa đáp ứng đủ nhu cầu crawl nên cần vào từng link sản phẩm bên trong thẻ li ở trong list đó, mỗi li chứa một sản phẩm. Dựa vào href của mỗi sản phẩm, mình sẽ gửi scrapy.Request tới url của sản phẩm và gọi **parse_macbook()** để crawl dữ liệu.
- **parse_macbook()**: Ở hàm này, sẽ tiến hành crawl những item mà đã được định nghĩa trong DemoCrawlItem(). Hãy xem xét kỹ website và inspect để trở đúng tới item đó trên DOM, ví dụ như khi shop hết hàng thì selector của giá sản phẩm sẽ khác, hoặc selector của giá sale cũng sẽ khác khi mua hàng online. Nên cân nhắc kỹ khi copy selector, đừng copy luôn vì có thể crawl không ra đúng data như yêu cầu.

Tiến hành crawl

Scrapy hỗ trợ xuất dữ liệu sang các định dạng khác nhau như JSON, CSV và XML.

```
$ scrapy crawl macbook_tgdd -o macbook_tgdd.json
```



```
tqkhai@khai-master: ~/demo_crawl/demo_crawl/spiders
'scrappy.downloadermiddlewares.httpcompression.HttpCompressionMiddleware',
'scrappy.downloadermiddlewares.redirect.RedirectMiddleware',
'scrappy.downloadermiddlewares.cookies.CookiesMiddleware',
'scrappy.downloadermiddlewares.httpproxy.HttpProxyMiddleware',
'scrappy.downloadermiddlewares.stats.DownloaderStats']
2025-03-02 10:30:13 [scrappy.middleware] INFO: Enabled spider middlewares:
['scrappy.spidermiddlewares.httperror.HttpErrorMiddleware',
'scrappy.spidermiddlewares.referer.RefererMiddleware',
'scrappy.spidermiddlewares.urllength.UrlLengthMiddleware',
'scrappy.spidermiddlewares.depth.DepthMiddleware']
2025-03-02 10:30:13 [scrappy.middleware] INFO: Enabled item pipelines:
[]
2025-03-02 10:30:13 [scrappy.core.engine] INFO: Spider opened
2025-03-02 10:30:13 [scrappy.extensions.logstats] INFO: Crawled 0 pages (at 0 pages/min), scraped 0 items (at 0 items/min)
2025-03-02 10:30:13 [scrappy.extensions.telnet] INFO: Telnet console listening on 127.0.0.1:6023
2025-03-02 10:30:13 [scrappy.core.engine] DEBUG: Crawled (403) <GET https://www.thegioididong.com/robots.txt> (referer: None) ['partial']
2025-03-02 10:30:13 [scrappy.core.engine] DEBUG: Crawled (403) <GET https://www.thegioididong.com/laptop-apple-macbook> (referer: None) ['partial']
2025-03-02 10:30:13 [scrappy.spidermiddlewares.httperror] INFO: Ignoring response <403 https://www.thegioididong.com/laptop-apple-macbook>: HTTP status code is not handled or not allowed
2025-03-02 10:30:13 [scrappy.core.engine] INFO: Closing spider (finished)
```

Kết quả là thu được lỗi 403. Do các website thường có cơ chế chặn bot crawl data. Phải xử lý ngay lỗi này bằng việc cấu hình thêm **User-Agent** trong **DEFAULT_REQUEST_HEADERS** mà Scrapy comment đi ở tập tin **settings.py** như sau, đồng thời cấu hình thêm **utf-8** để tránh lỗi font:

```
39 # Override the default request headers:
40 DEFAULT_REQUEST_HEADERS = {
41     "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
42     "Accept-Language": "en",
43     "User-Agent": "Mozilla/5.0 (X11; Linux x86_64; rv:48.0) Gecko/20100101 Firefox/48.0",
44 }
45
46 FEED_EXPORT_ENCODING = 'utf-8'
47
```

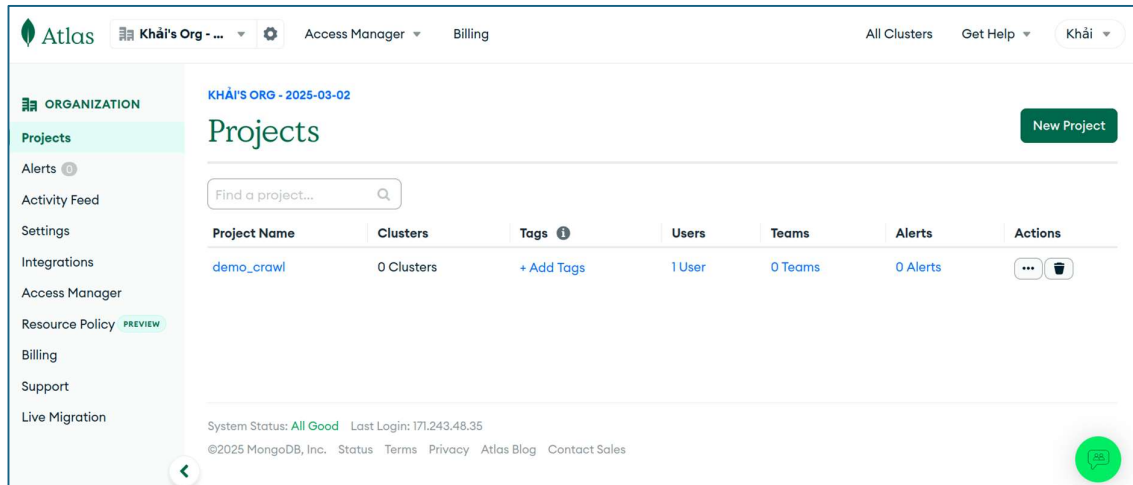
3. KẾT NỐI VÀ ĐĂNG DỮ LIỆU LÊN MONGODB ATLAS

Cài đặt pymongo

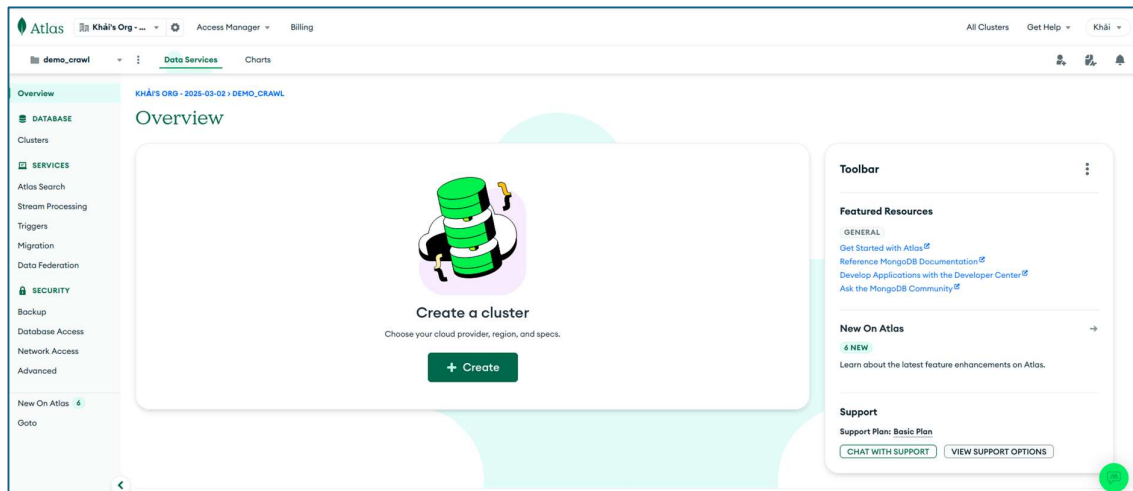
\$ python3 -m pip install pymongo

Lấy chuỗi kết nối

- Tạo tài khoản trên <https://account.mongodb.com/account/login>
- Vào mục Projects → tạo project **demo_crawl**



- Vào demo_crawl → chọn Create (cluster)



Deploy your cluster

Use a template below or set up advanced configuration options. You can also edit these configuration options once the cluster is created.

☐ **M10** **\$0.10/hour**

Dedicated cluster for development environments and low-traffic applications.

STORAGE	RAM	vCPU
10 GB	2 GB	2 vCPUs

☐ **Flex** **From \$0.011/hour**
Up to \$30/month

For application development and testing, with on-demand burst capacity for unpredictable traffic.

STORAGE	RAM	vCPU
5 GB	Shared	Shared

☒ **Free**

For learning and exploring MongoDB in a cloud environment.

STORAGE	RAM	vCPU
512 MB	Shared	Shared

☒ **Free forever!** Your free cluster is ideal for experimenting in a limited sandbox. You can upgrade to a production cluster anytime.

Configurations

Name
You cannot change the name once the cluster is created.

Cluster0

Provider

aws

Google Cloud

Azure

Region

Hong Kong (ap-east-1) ★

★ Recommended ⓘ Low carbon emissions ⓘ

Tag (optional)
Create your first tag to categorize and label your resources; more tags can be added later. [Learn more.](#)

☐ Automate security setup ⓘ

☐ Preload sample dataset ⓘ

I'll do this later

Go to Advanced Configuration

Create Deployment

Thêm địa chỉ IP hiện tại của máy chạy crawl data vào cluster vừa tạo và thiết lập tài khoản user cho database.

Connect to Cluster0

1

2

3

Set up connection securityChoose a connection methodConnect

You need to secure your MongoDB Atlas cluster before you can use it. Set which users and IP addresses can access your cluster now. [Read more](#)

1. Add a connection IP address

Add Your Current IP Address

Add a Different IP Address

Allow Access from Anywhere

2. Create a database user

This first user will have [atlasAdmin](#) permissions for this project.

You'll need your database user's credentials in the next step. Copy the database user password.

Username

Password

ex. dbUser

ex. dbUserPassword

HIDE

Copy

Create Database User

Close

Choose a connection method

Connect to Cluster0

1

2

3

Set up connection securityChoose a connection methodConnect

You need to secure your MongoDB Atlas cluster before you can use it. Set which users and IP addresses can access your cluster now. [Read more](#)

1. Add a connection IP address

✓ Your current IP address (171.243.48.35) has been added to enable local connectivity. Only an IP address you add to your Access List will be able to connect to your project's clusters. Add more later in [Network Access](#).

2. Create a database user

✓ A database user has been added to this project. Create another user later in [Database Access](#).

You'll need your database user's credentials in the next step.

Close

Choose a connection method

Chọn **Choose a connection method**

Connect to Cluster0

1Set up connection security

2Choose a connection method

3Connect

Connect to your application

**Drivers**
Access your Atlas data using MongoDB's native drivers (e.g. Node.js, Go, etc.)



Access your data through tools

**Compass**
Explore, modify, and visualize your data with MongoDB's GUI

**Shell**
Quickly add & update data using MongoDB's Javascript command-line interface

**MongoDB for VS Code**
Work with your data in MongoDB directly from your VS Code environment

**Atlas SQL**
Easily connect SQL tools to Atlas for data analysis and visualization

Go Back

Close

Connect to Cluster0

✓

✓

3

Set up connection security

Choose a connection method

Connect

Connecting with MongoDB Driver

1. Select your driver and version

We recommend installing and using the latest driver version.

Driver

Version

Python

4.7 or later

2. Install your driver

Run the following on the command line

Note: Use appropriate Python 3 executable

```
python -m pip install "pymongo[srv]"
```

[View MongoDB Python Driver installation instructions.](#)

3. Add your connection string into your application code

Use this connection string in your application

☐ View full code sample

```
mongodb+srv://admin:<db_password>@cluster0.rng21.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0
```

Replace `<db_password>` with the password for the `admin` database user. Ensure any option params are [URL encoded](#).

RESOURCES

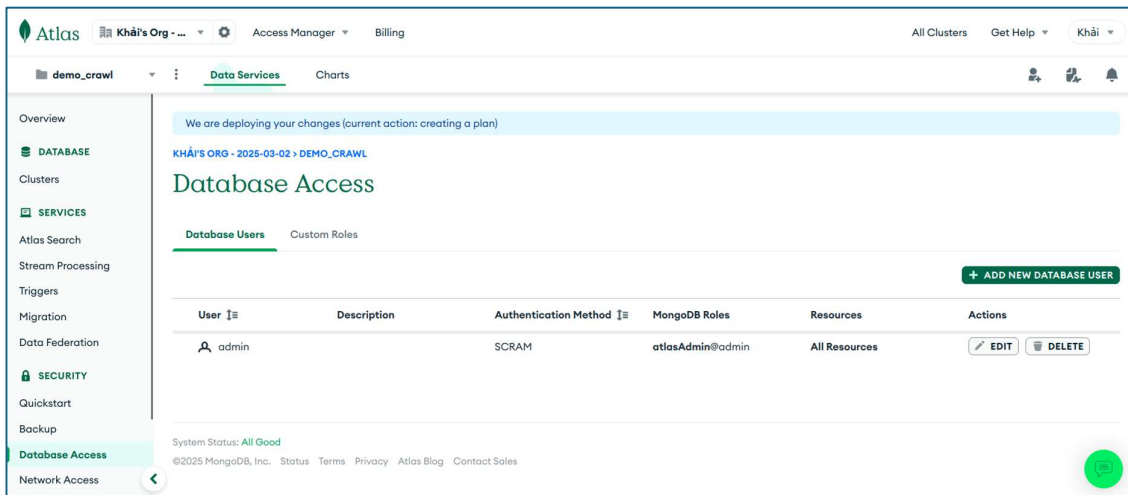
[Get started with the Python Driver](#)[Python Starter Sample App](#)

[Access your Database Users](#)[Troubleshoot Connections](#)

Go Back

Done

➤ Cài đặt người dùng truy cập → **Data Access** → **Add New Database User**



Add New Database User

Create a database user to grant an application or user access to databases and collections in your clusters in this Atlas project. Granular access control can be configured with default privileges or custom roles. You can grant access to an Atlas project or organization using the corresponding [Access Manager](#)

Authentication Method

Password

Certificate

AWS IAM

Federated Auth (MongoDB 7.0 and up)

MongoDB uses [SCRAM](#) as its default authentication method.

Password Authentication

[SHOW](#)

[Autogenerate Secure Password](#) [Copy](#)

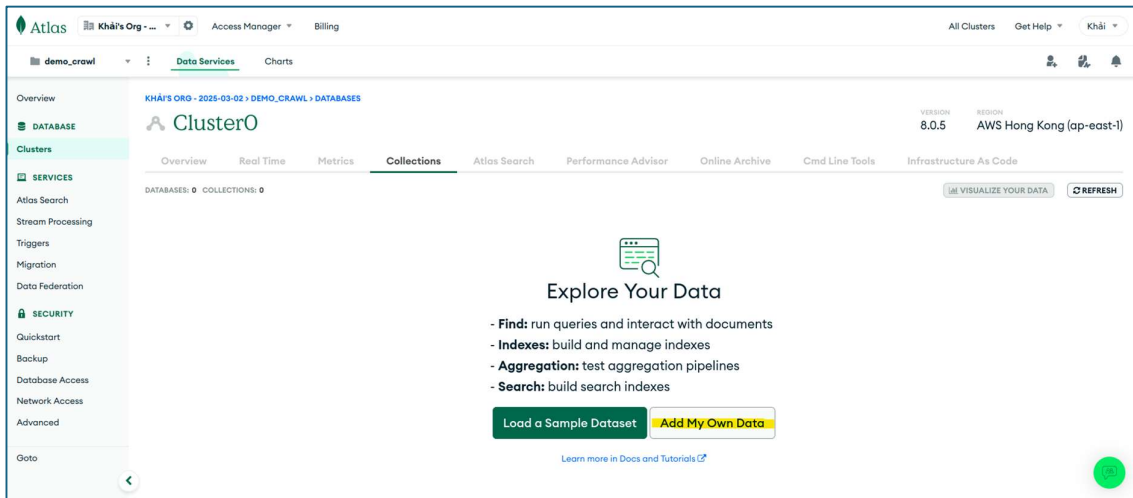
User Description

Add an optional description to your user.

Chọn Password → điền thông tin username và password → Chọn Add User (cuối form)

➤ Tiến hành tạo database

Vào cluster → chọn Collections → Chọn Add My Own Data



Điền Database name là tên CSDL và Collection name có thể coi như tên bảng.

×

Create Database

Database name ?

Collection name ?

Additional Preferences

Select ▼

Cancel

Create

- Tiến hành thiết lập Pipeline để xử lý việc đẩy dữ liệu lên MongoDB Atlas

Xây dựng pipeline kết nối MongoDB Atlas và đẩy dữ liệu vào database

```
from itemadapter import ItemAdapter
from scrapy.exceptions import DropItem
import pymongo

class MongoDBPipeline:
    def __init__(self):
        self.client =
pymongo.MongoClient('mongodb+srv://<username>:<password>@cluster0.rng21.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0')
        self.db = self.client["LAPTOPDB"]

    def process_item(self, item, spider):
        collection = self.db["MACBOOK"]
        try:
            collection.insert_one(dict(item))
        except Exception as e:
            raise DropItem(f"Error in pipeline: {e}")
        return item

class JsonDBPipeline:
    def process_item(self, item, spider):
        self.file = open('data.json', 'a', encoding='utf-8')
        line = json.dumps(dict(item), ensure_ascii=False) + "\n"
        self.file.write(line)
        self.file.close()
        return item
```

Mở cấu hình pipeline **ITEM_PIPELINES** trong tập tin **settings.py** bằng việc bỏ chú thích và chỉ định rõ class xử lý cho MongoDB được xây dựng trong tập tin **pipelines.py**

```

66 # Configure item pipelines
67 # See https://docs.scrapy.org/en/latest/topics/item-pipeline.html
68 ITEM_PIPELINES = {
69     "demo_crawl.pipelines.MongoDBPipeline": 300,
70 }

```

➤ Chạy lại lệnh crawl data và xem kết quả

\$ scrapy crawl macbook_tgdd -o macbook_tgdd.json

LAPTOPDB.MACBOOK

STORAGE SIZE: 8KB LOGICAL DATA SIZE: 1.97KB TOTAL DOCUMENTS: 19 INDEXES TOTAL SIZE: 8KB

Find Indexes Schema Anti-Patterns 0 Aggregation Search Indexes

Generate queries from natural language in Compass

Filter Type a query: { field: 'value' }

QUERY RESULTS: 1-19 OF 19

<pre> _id: ObjectId('67c463624caf2e4723488655') product_name: "Laptop Apple MacBook Air 13 inch M1 8GB/256GB (MGN63SA/A)" price: null </pre>
<pre> _id: ObjectId('67c463634caf2e4723488656') product_name: "Laptop Macbook Air 13 inch M2 16GB/512GB Sạc 70W" price: "29.190.000đ " </pre>
<pre> _id: ObjectId('67c463634caf2e4723488657') product_name: "Laptop Apple MacBook Pro 14 inch Nano M4 Max 36GB/1TB" price: "82.090.000đ " </pre>

4. BÀI TẬP

Bài tập 01: thực hiện thu thập dữ liệu từ website "Cổng dịch vụ công quốc gia". Kết quả là một bộ data (câu hỏi, câu trả lời) dưới dạng json. Sau đó, đổ dữ liệu thô và dữ liệu đã tiền xử lý vào MongoDB Atlas.

Link: <https://dichvucong.gov.vn/p/home/dvc-cau-hoi-pho-bien.html>

Bài tập 02: truy cập vào trang <https://www.vietnamworks.com/it-kv> để xem danh sách các mẫu tin tuyển dụng. Thực hiện các yêu cầu sau:

- Định nghĩa cấu trúc thích hợp để lưu trữ đối tượng thông tin tuyển dụng
- Thu thập dữ liệu từ website trên
- Đổ dữ liệu thô và dữ liệu đã tiền xử lý vào MongoDB Atlas.

Bài tập 03: xử lý 2 bài tập trên thông qua một project

--- Hết ---

TÍCH HỢP SCRAPY & SELENIUM QUA SCRAPY-SELENIUM

Selenium là một bộ kiểm tra tự động (mã nguồn mở) miễn phí dành cho các ứng dụng web trên nhiều trình duyệt và nền tảng khác nhau. Chủ yếu được sử dụng để tự động hóa việc kiểm tra các ứng dụng web, tuy nhiên nhưng không giới hạn ở điều đó. Selenium được hỗ trợ bởi tất cả các nhà cung cấp trình duyệt lớn như chrome, firefox, ie, safari... để biến Selenium trở thành phần của trình duyệt. Nó cũng là công nghệ cốt lõi của vô số các công cụ, API và framework tự động hóa.

Scrapy là một trong những nền tảng web scraping phổ biến nhất hiện nay. Tuy nhiên, vẫn còn một số hạn chế không thể hiển thị các trang web được tải JavaScript hoặc những trang có tính năng chặn bot.

Scrapy-selenium là một phần mềm trung gian Scrapy chuyển hướng các yêu cầu Scrapy vào trình điều khiển Selenium. Điều này cho phép thu thập các trang web động bằng Scrapy cùng với các tính năng tự động hóa trình duyệt không đầu khác, chẳng hạn như:

- Đợi thời gian hoặc phần tử cụ thể.
- Nhấp vào các nút và điền vào biểu mẫu.
- Chụp ảnh màn hình.
- Thực thi mã JavaScript tùy chỉnh.

Cài đặt scrapy-selenium

```
$ pip3 install scrapy-selenium
```

Cài đặt các thư viện cần thiết

```
$ pip3 install scrapy-selenium selenium scrapy webdriver-manager
```

Xem thông tin thư mục cài đặt site-packages

```
$ pip3 show scrapy-selenium
```

```
khaitq@khaitq:~$ pip3 show scrapy-selenium
Name: scrapy-selenium
Version: 0.0.7
Summary: Scrapy with selenium
Home-page: https://github.com/clemfromspace/scrapy-selenium
Author: UNKNOWN
Author-email: UNKNOWN
License: UNKNOWN
Location: /home/khaitq/.local/lib/python3.10/site-packages
Requires: scrapy, selenium
Required-by:
khaitq@khaitq:~$
```

Di chuyển đến thư mục trong phần location để thay đổi tập tin middlewares.py

```
$ cd /home/khaitq/.local/lib/python3.10/site-packages
```

```
$ cd scrapy_selenium
```

```
$ vi middlewares.py
```

```
"""This module contains the ``SeleniumMiddleware`` scrapy middleware"""

from importlib import import_module

from scrapy import signals
from scrapy.exceptions import NotConfigured
from scrapy.http import HtmlResponse
from selenium.webdriver.support.ui import WebDriverWait

from .http import SeleniumRequest

class SeleniumMiddleware:
    """Scrapy middleware handling the requests using selenium"""

    def __init__(self, driver_name, driver_executable_path,
                  browser_executable_path, command_executor, driver_arguments):
        """Initialize the selenium webdriver

        Parameters
        -----
        driver_name: str
            The selenium ``WebDriver`` to use
        driver_executable_path: str
            The path of the executable binary of the driver
        driver_arguments: list
            A list of arguments to initialize the driver
        browser_executable_path: str
            The path of the executable binary of the browser
        command_executor: str
            Selenium remote server endpoint
        """

        webdriver_base_path = f'selenium.webdriver.{driver_name}'

        driver_class_module = import_module(f'{webdriver_base_path}.webdriver')
        driver_class = getattr(driver_class_module, 'WebDriver')
```

```

driver_options_module = import_module(f'{webdriver_base_path}.options')
driver_options_klass = getattr(driver_options_module, 'Options')

driver_options = driver_options_klass()

if browser_executable_path:
    driver_options.binary_location = browser_executable_path
    for argument in driver_arguments:
        driver_options.add_argument(argument)

driver_kwargs = {
    'executable_path': driver_executable_path,
    f'{driver_name}_options': driver_options
}

# locally installed driver
if driver_executable_path is not None:
    driver_kwargs = {
        'executable_path': driver_executable_path,
        f'{driver_name}_options': driver_options
    }
    self.driver = driver_klass(**driver_kwargs)
# remote driver
elif command_executor is not None:
    from selenium import webdriver
    capabilities = driver_options.to_capabilities()
    self.driver = webdriver.Remote(command_executor=command_executor,
                                   desired_capabilities=capabilities)
# webdriver-manager
else:
    # selenium4+ & webdriver-manager
    from selenium import webdriver
    from webdriver_manager.chrome import ChromeDriverManager
    from selenium.webdriver.chrome.service import Service as ChromeService
    if driver_name and driver_name.lower() == 'chrome':
        # options = webdriver.ChromeOptions()
        # options.add_argument(o)
        self.driver = webdriver.Chrome(options=driver_options,
                                       service=ChromeService(ChromeDriverManager().install()))

    @classmethod
    def from_crawler(cls, crawler):

```

```

        """Initialize the middleware with the crawler settings"""

        driver_name = crawler.settings.get('SELENIUM_DRIVER_NAME')
        driver_executable_path = crawler.settings.get('SELENIUM_DRIVER_EXECUTABLE_PATH')
        browser_executable_path = crawler.settings.get('SELENIUM_BROWSER_EXECUTABLE_PATH')
        command_executor = crawler.settings.get('SELENIUM_COMMAND_EXECUTOR')
        driver_arguments = crawler.settings.get('SELENIUM_DRIVER_ARGUMENTS')

        if driver_name is None:
            raise NotConfigured('SELENIUM_DRIVER_NAME must be set')

        # let's use webdriver-manager when nothing specified instead | RN just for Chrome
        if (driver_name.lower() != 'chrome') and (driver_executable_path is None and command_executor is None):
            raise NotConfigured('Either SELENIUM_DRIVER_EXECUTABLE_PATH '
                                'or SELENIUM_COMMAND_EXECUTOR must be set')

        middleware = cls(
            driver_name=driver_name,
            driver_executable_path=driver_executable_path,
            browser_executable_path=browser_executable_path,
            command_executor=command_executor,
            driver_arguments=driver_arguments
        )

        crawler.signals.connect(middleware.spider_closed, signals.spider_closed)

        return middleware

    def process_request(self, request, spider):
        """Process a request using the selenium driver if applicable"""

        if not isinstance(request, SeleniumRequest):
            return None

        self.driver.get(request.url)

        for cookie_name, cookie_value in request.cookies.items():
            self.driver.add_cookie(
                {
                    'name': cookie_name,
                    'value': cookie_value
                }
            )

```

```

    )

    if request.wait_until:
        WebDriverWait(self.driver, request.wait_time).until(
            request.wait_until
        )

    if request.screenshot:
        request.meta['screenshot'] = self.driver.get_screenshot_as_png()

    if request.script:
        self.driver.execute_script(request.script)

    body = str.encode(self.driver.page_source)

    # Expose the driver via the "meta" attribute
    request.meta.update({'driver': self.driver})

    return HtmlResponse(
        self.driver.current_url,
        body=body,
        encoding='utf-8',
        request=request
    )

def spider_closed(self):
    """Shutdown the driver when spider is closed"""

    self.driver.quit()

```

Khởi tạo project với tên dvc cho trang <https://dichvucong.gov.vn/p/home/dvc-cau-hoi-pho-bien.html>

```
$ scrapy startproject demo_dvc demo_scrapy_selenium
```

```
khaitq@khaitq:~/demo_scrapy_selenium$ tree
.
├── demo_dvc
│   ├── __init__.py
│   ├── items.py
│   ├── middlewares.py
│   ├── pipelines.py
│   ├── settings.py
│   └── spiders
│       └── __init__.py
└── scrapy.cfg

2 directories, 7 files
khaitq@khaitq:~/demo_scrapy_selenium$
```

Cấu hình trong tập tin settings.py

```
SELENIUM_DRIVER_NAME = 'chrome'
SELENIUM_DRIVER_EXECUTABLE_PATH = None # webdriver-manager will manage it by itself
SELENIUM_DRIVER_ARGUMENTS=[] # change it to ['-headless'] run in headless mode

DOWNLOADER_MIDDLEWARES = {
    'scrapy_selenium.SeleniumMiddleware': 800
}
```

Tạo tập tin Spider

```
$ cd /home/khaitq/demo_scrapy_selenium/demo_dvc/spiders
```

```
$ scrapy genspider dvc https://dichvucong.gov.vn/p/home/dvc-cau-hoi-pho-bien.html
```

Tạo đối tượng items.py

```
class DemoDvcItem(scrapy.Item):
    # define the fields for your item here like:
    que = scrapy.Field()
    ans = scrapy.Field()
```

Xây dựng spider lấy dữ liệu trên dvc.py

```
import scrapy
from demo_dvc.items import DemoDvcItem
from scrapy_selenium import SeleniumRequest

class DvcSpider(scrapy.Spider):
    name = "dvc"
    allowed_domains = ["dichvucong.gov.vn"]
    start_urls = ["https://dichvucong.gov.vn/p/home/dvc-cau-hoi-pho-bien.html"]
```

```

def start_requests(self):
    url = "https://dichvucong.gov.vn/p/home/dvc-cau-hoi-pho-bien.html"
    yield SeleniumRequest(
        url=url,
        callback=self.parse
    )

def parse(self, response):
    # Request tới từng sản phẩm có trong danh sách các Macbook dựa vào href
    for item_url in response.css("div.list-document > a.item ::attr(href)").extract():
        print(item_url)
        # Nếu có sản phẩm thì sẽ gọi tới function parse_macbook
        yield SeleniumRequest(url=response.urljoin(item_url), callback=self.parse_dvc,
            wait_time=10)

    # nếu có sản phẩm kế tiếp thì tiếp tục crawl
    next_page = response.css("li.next > a ::attr(href)").extract_first()

    if next_page:
        yield SeleniumRequest(url=response.urljoin(next_page), callback=self.parse)

def parse_dvc(self, response):
    item = DemoDvcItem()
    item['que'] = response.css('div.cau-hoi-pho-bien > h1.main-title-sub ::text').extract_first()
    item['ans'] = response.css('div.article > p:nth-child(3) ::text').extract_first()

    yield item

```

Tiến hành lấy dữ liệu

\$ scrapy crawl dvc --output data.json